

Análisis de Algoritmos 2022/2023

Práctica 2

Íñigo Álvarez Negueruela, 1271

Código	Gráficas	Memoria	Total

1. Introducción.

Vamos a realizar una práctica en la que implementaremos varias funciones y algoritmos y calcularemos su complejidad. Se programará en el lenguaje C y además se usará gnuplot para generar gráficas, midiendo tiempos y comparando datos.

2. Objetivos

El objetivo de esta práctica es aprender a implementar algunos algoritmos, además de medir tiempos y la complejidad de estas funciones. Además aprenderemos a generar gráficas con gnuplot comparando dichos tiempos con su supuesto resultado.

2.1 Apartado 1

El objetivo del apartado 4 es ordenar mediante el algoritmo Mergesort un array que hemos creado desordenado de los números de 1 a N mediante la función del apartado 3 de la práctica 1. Con ello aprenderemos el funcionamiento del Mergesort y contaremos el número de veces que se ejecuta la OB para ver si coincide con lo que debería. Además lo comprobamos con el exercise 4 de la práctica 1 para ver si funciona correctamente.

2.2 Apartado 2

En este apartado probaremos nuestra función del apartado anterior con el exercise5 de la práctica 1 para medir así los tiempos de ejecución y calcular las veces que se repite la operación básica para un N determinado e imprimiendo el caso mejor peor y la media que nos haya salido.

2.3 Apartado 3

El objetivo del apartado 4 es ordenar mediante el algoritmo Quicksort un array que hemos creado desordenado de los números de 1 a N mediante la función del apartado 3 de la práctica 1. Con ello aprenderemos el funcionamiento del Quicksort y contaremos el número de veces que se ejecuta la OB para ver si coincide con lo que debería. Además lo comprobamos con el exercise 4 de la práctica 1 para ver si funciona correctamente. Esto lo haremos cogiendo el pivote como el primer elemento del array.

2.4 Apartado 4

En este apartado probaremos nuestra función del apartado anterior con el exercise5 de la práctica 1 para medir así los tiempos de ejecución y calcular las veces que se repite la operación básica para un N determinado e imprimiendo el caso mejor peor y la media que nos haya salido.

2.5 Apartado 5

El objetivo del apartado 5 es ver como cambia el tiempo de ejecución y las obs dependiendo del pivote que elijas. Primero nos pide que elijamos el del medio, pero

debido a la aleatoriedad del array inicial este no debería cambiar la gráfica. Sin embargo, comparando entre el 1 el del medio y el último será más óptimo que cogiendo uno al azar.

3. Herramientas y metodología

3.1 Apartado 1

Hemos utilizado el entorno de desarrollo Linux, con Visual Studio Code como herramienta para programar. Para compilar el programa hemos utilizado gcc así como algunas flags como -Wall, -ansi o -pedantic. Valgrind lo hemos empleado para comprobar si la memoria de nuestro programa estaba correctamente usada. Para comprobar el funcionamiento de este apartado hemos usado el test que usamos la anterior práctica anterior (en el Makefile), cambiando sus datos para comprobar posibles fallos y cambiando el exercise4.c para que esta vez use el MergeSort. Aparte de todo esto hemos usado algún main propio como prueba.

3.2 Apartado 2

Como en el apartado anterior también hemos utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Gnuplot ha sido necesario para generar una gráfica con los números random generados y su número de veces. Para comprobar el funcionamiento hemos modificado el Makefile y el exercise5.c. Con las gráficas generadas también hemos podido comprobar si funcionaba correctamente.

3.3 Apartado 3

En este apartado también se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Como pruebas hemos usado el test que hay en el Makefile (exercise4_test) cambiando los datos en función de las pruebas que quisiéramos hacer, ya que en este caso usamos el algoritmo quickSort.

3.4 Apartado 4

En el apartado 4 también se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Además, hemos empleado la herramienta Gnuplot para generar gráficas con los datos obtenidos. La prueba principal ha sido el test del makefile (exercise5_test) cambiando los datos para generar distintas pruebas.

3.5 Apartado 5

En este último apartado se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. Para comprobar nuestra función hemos modificado las pruebas del apartado 4 y debuggeado con algún printf los valores que iba tomando el pivote en función del primer elemento, el último y el del medio.

4. Código fuente

Aquí ponéis el código fuente **exclusivamente de las rutinas que habéis desarrollado vosotros** en cada apartado.

4.1

```
int mergesort(int* tabla, int ip, int iu)
{
    int mid = 0;
    int count = 0;
    int c = 0;

    if (!tabla || iu < ip || ip < 0)
        return ERR;
    if (ip == iu)
        return OK;
    mid = (iu+ip)/2;
    c = mergesort(tabla, ip, mid);
    if (c == ERR)
        return ERR;
    count += c;
    c = mergesort(tabla, mid+1, iu);
    if (c == ERR)
        return ERR;
    count += c;
    c = merge(tabla, ip, iu, mid);
    if (c == ERR)
        return ERR;
    count += c;
    return count;
}
```

```

int merge(int* tabla, int ip, int iu, int imedio)
{
    int *aux;
    int i, j, k;
    int count = 0;

    if (!tabla || ip<0 || iu<ip || imedio<ip || iu<imedio)
        return ERR;
    aux = malloc(sizeof(int*) * (iu-ip)+1);
    if (aux == NULL)
        return ERR;
    i = ip;
    j = imedio+1;
    k = 0;
    while (i <= imedio && j <= iu)
    {
        if (tabla[i] < tabla[j])
        {
            aux[k]=tabla[i];
            i++;
        }
        else
        {
            aux[k] = tabla[j];
            j++;
        }
        count++;
        k++;
    }
    if (i > imedio)
    {
        while (j <= iu)
        {
            aux[k] = tabla[j];
            j++;
            k++;
        }
    }
    else if(j > iu)
    {
        while (i <= imedio)
        {
            aux[k] = tabla[i];
            i++;
            k++;
        }
    }
    i = ip;
    j = iu;
    k = 0;
    while (i <= j)
    {
        tabla[i] = aux[k];
        i++;
        k++;
    }

    free(aux);
    return count;
}

```

4.3 Apartado 3

```

int quicksort(int* tabla, int ip, int iu){
    int M=0, pos=0, counter=0, ob=0;
    if(ip>iu || !tabla || ip<0){
        return ERR;
    }
    if(ip==iu){
        return 0;
    }
    else{
        counter=partition(tabla,ip,iu, &pos);
        if(counter==ERR){
            return ERR;
        }
        ob+=counter;
        M=pos;
        if (ip<M-1){
            counter=quicksort(tabla,ip,M-1);
            if(counter==ERR){
                return ERR;
            }
            ob+=counter;
        }
        if (M+1 < iu){
            counter=quicksort(tabla,M+1,iu);
            if(counter==ERR){
                return ERR;
            }
            ob+=counter;
        }
    }
    return ob;
}

```

```

int partition(int* tabla, int ip, int iu,int *pos){
    int k,M,i, swap, counter=0;
    if(ip<0 || iu<ip || !tabla || !pos){
        return ERR;
    }
    counter+= median(tabla,ip, iu, pos);
    if(counter==ERR){
        return ERR;
    }
    M=(*pos);
    k= tabla[M];
    swap= tabla[ip];
    tabla[ip]= tabla[M];
    tabla[M]=swap;

    M=ip;
    for(i=ip+1; i<=iu; i++){
        if (tabla[i]<k){
            M++;
            swap= tabla[i];
            tabla[i]= tabla[M];
            tabla[M]=swap;
        }
        counter++;
    }
    swap= tabla[ip];
    tabla[ip]= tabla[M];
    tabla[M]=swap;
    (*pos)=M;
    return counter;
}

```

```

int median(int *tabla, int ip, int iu,int *pos){
    if(ip<0 || iu<ip || !tabla || !pos){
        return ERR;
    }
    (*pos)=ip;
    return 0;
}

```

4.5 Apartado 5

```
int median_avg(int *tabla, int ip, int iu, int *pos){  
    if(ip<0 || iu<ip || !tabla || !pos){  
        return ERR;  
    }  
  
    (*pos)=(ip + iu)/2;  
    return 0;  
}
```

```
int median_stat(int *tabla, int ip, int iu, int *pos){  
    if(ip<0 || iu<ip || !tabla || !pos){  
        return ERR;  
    }  
  
    int m=0;  
    m=(ip + iu)/2;  
    if((tabla[ip]<tabla[iu] && tabla[ip]>tabla[m]) || (tabla[ip]>tabla[iu] && tabla[ip]<tabla[m])){  
        *pos=ip;  
        return 4;  
    }  
    else if((tabla[m]<tabla[iu] && tabla[m]>tabla[ip]) || (tabla[m]>tabla[iu] && tabla[m]<tabla[ip])){  
        *pos=m;  
    }  
    else{  
        *pos=iu;  
    }  
    return 8;  
}
```

5. Resultados, Gráficas

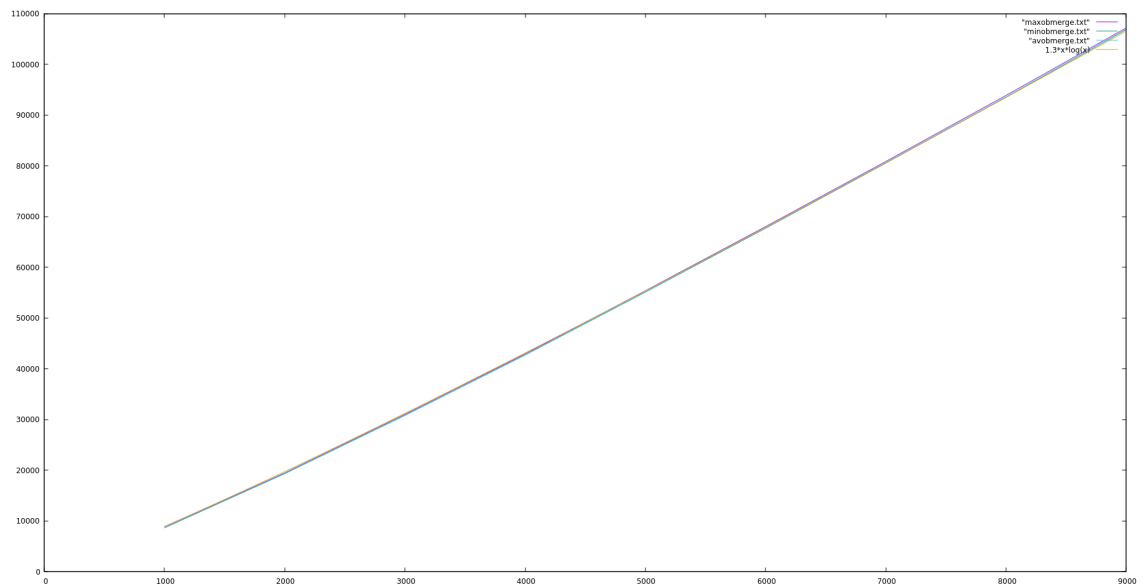
5.1 Apartado 1

En este apartado, se genera un array desordenado de un size determinado llamando a la función del apartado 3. Tras esto el exercise4.c llama a MergeSort para ordenar de menor a mayor este array. Por ejemplo, si el size es 10, aparecerá en la pantalla los números ordenados del 0 al 9 (de 0 a size-1).

5.2 Apartado 2

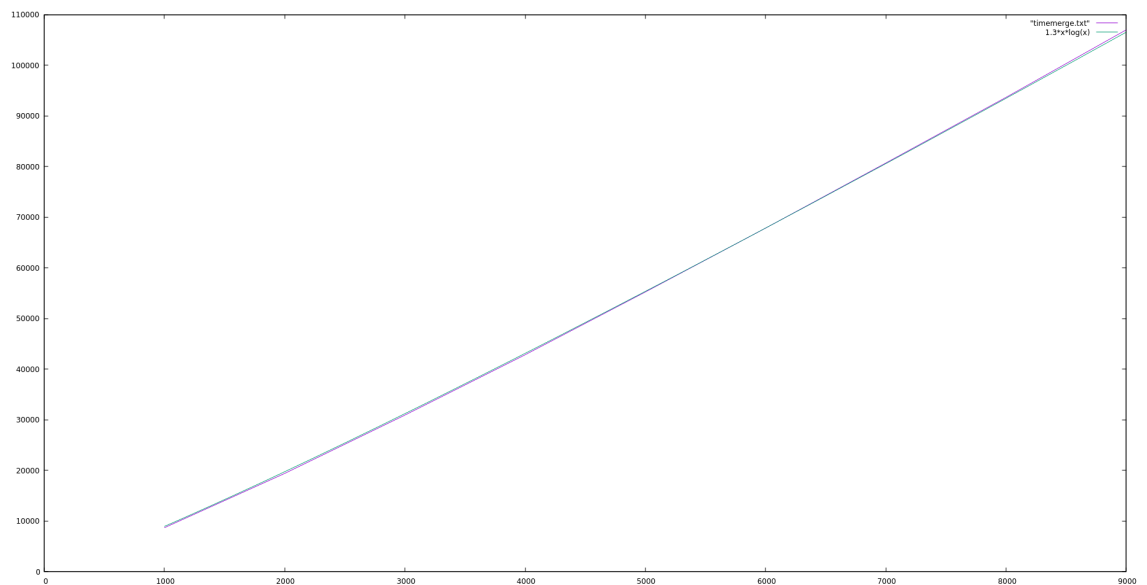
El apartado 2, devuelve un fichero en el cuál mide para cada array desordenado de tamaño size estos datos tras la llamada a MergeSort: el tiempo de ejecución del algoritmo, las veces que se ha ejecutado la operación básica para los casos mejor y peor y la media entre estos dos últimos datos. Este lo hace para sizes entre el num_min y el num_máx con un incremento incr. Con estos datos podemos concluir que MergeSort es un gran algoritmo, hay una diferencia muy pequeña entre el caso peor, el caso medio y el caso mejor, como se refleja en la gráfica reflejada a continuación.

Gráfica comparando los tiempos mejor peor y medio en OBs para MergeSort, comentarios a la gráfica:



Aunque no se aprecia apenas diferencia, la gráfica del caso mejor está por debajo de la del caso medio que está por debajo del caso peor. Pero la diferencia es muy pequeña en este algoritmo como se ha explicado anteriormente. La complejidad es $O(n \log n)$

Gráfica con el tiempo medio de reloj para MergeSort, comentarios a la gráfica:



El tiempo medio también tiene la misma complejidad y al compararlo con $x \log(x)$ (lo hemos tenido que ajustar) vemos que nos da bien.

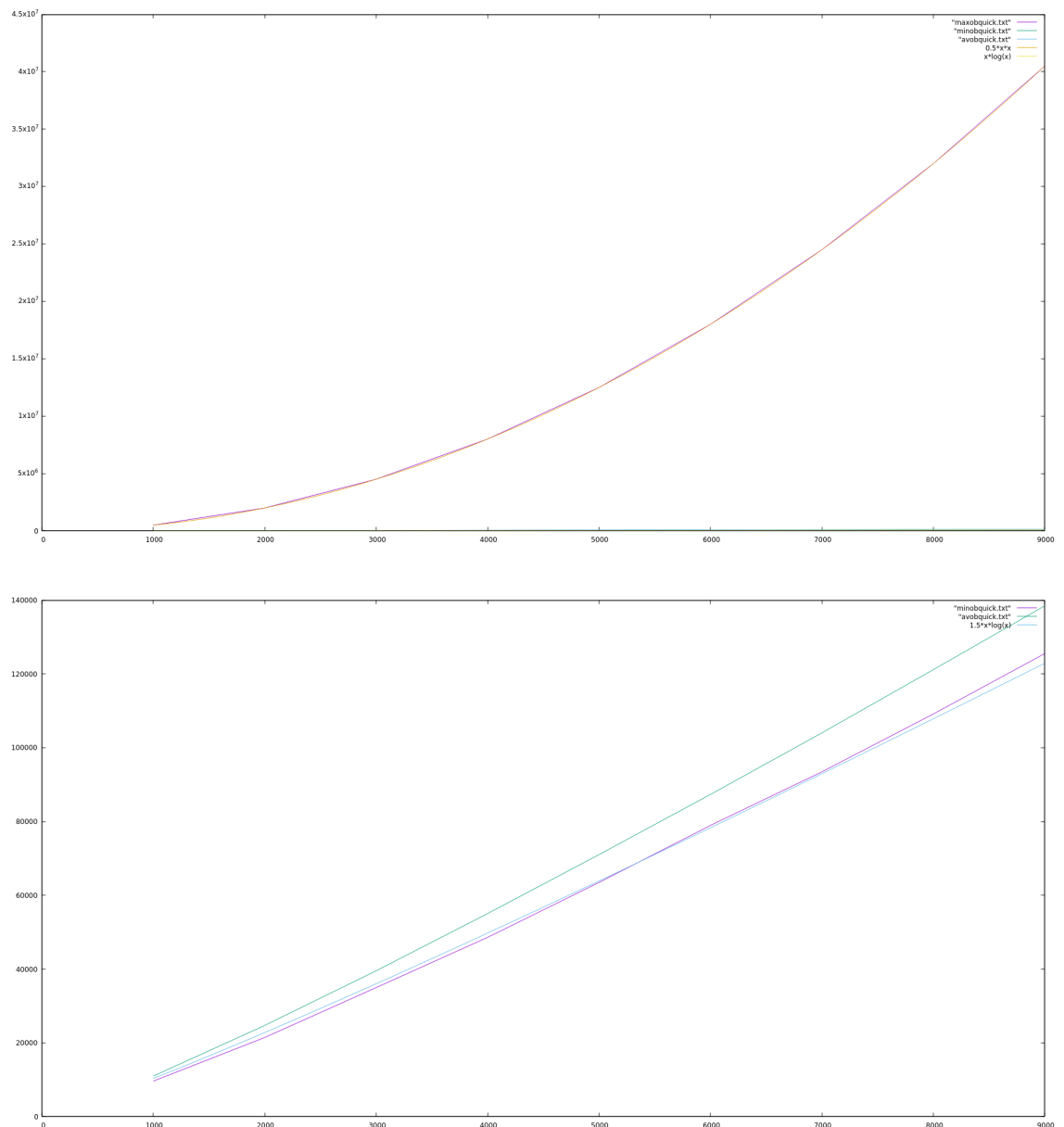
5.3 Apartado 3

Para el apartado 3 generamos un array desordenado de un size determinado llamando a la función del apartado 3. Tras esto el exercise4.c llama a QuickSort la cual ordena de menor a mayor este array. Por ejemplo, si el size es 10, aparecerá en la pantalla los números ordenados del 0 al 9 (de 0 a size-1).

5.4 Apartado 4

El apartado 2, devuelve un fichero en el cuál mide para cada array desordenado de tamaño size estos datos tras la llamada a QuickSort: el tiempo de ejecución del algoritmo, las veces que se ha ejecutado la operación básica para los casos mejor y peor y la media entre estos dos últimos datos. Este lo hace para sizes entre el num_min y el num_máx con un incremento incr. Viendo las gráficas podemos decir que no es un buen algoritmo ya que el peor caso dispara en exceso las comparaciones.

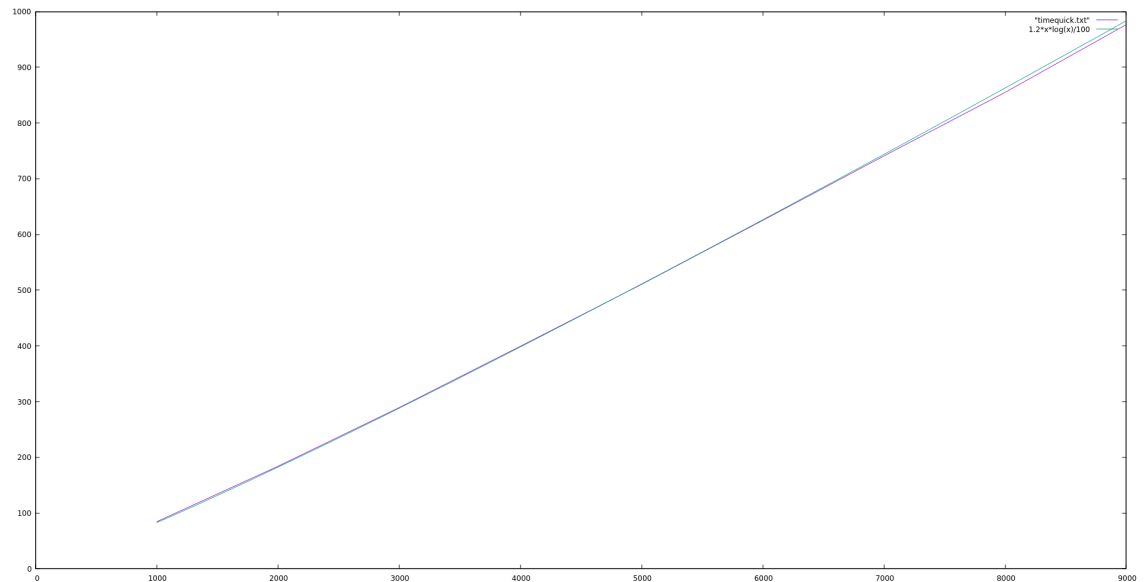
Gráfica comparando los tiempos mejor peor y medio en OBs para QuickSort, comentarios a la gráfica:



Esta gráfica la hemos dividido en 2 imágenes ya que el caso peor se encuentra muy por encima de las otras gráficas y no se aprecia la curva de las otras 3 si queremos que se vea esta. Como podemos ver, tanto el caso mejor como el medio se asemejan a la función $n \cdot \log(n)$, a parte hemos tenido que hacer un

ajuste y multiplicarla por 1.5. A su vez, el caso peor es similar a $n^2/2$, es por eso que tener una posibilidad de que un caso sea tan alto hace que el algoritmo no sea eficiente.

Gráfica con el tiempo medio de reloj para QuickSort, comentarios a la gráfica:

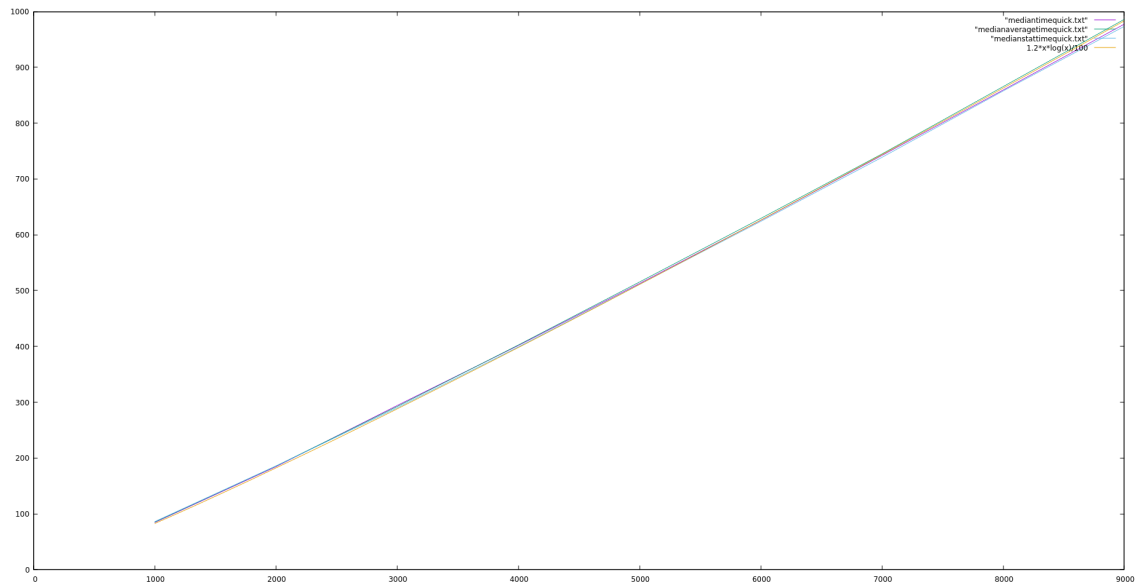


El tiempo medio también tiene la misma complejidad y al compararlo con $x * \log(x)$ vemos que nos da bien. Hemos dividido entre 100 ya que el tiempo dado es dividido entre 100 y hemos multiplicado por 1.2 para ajustarla.

5.5 Apartado 5

Resultados del apartado 5.

Gráfica con el tiempo medio de reloj comparando las versiones de Quicksort con las funciones pivote **median**, **median_avg** y **median_stat**:



Como vemos todas las curvas se ajustan a $n \cdot \log(n)$ debido a la aleatoriedad de las permutaciones, que hace que de igual que cojas siempre el elemento del medio o el primero. También se puede ver la gráfica del median_stat un poco por debajo de las otras dos, gracias a la elección la mediana entre el primero, último y el del medio. Hemos tenido que multiplicar $x \cdot \log(x)$ por 1.2 para ajustar esta gráfica.

5. Respuesta a las preguntas teóricas.

Aquí respondéis a las preguntas teóricas que se os han planteado en la práctica.

5.1 Pregunta 1

MergeSort: Teóricamente la complejidad de este algoritmo debería ser la misma que empíricamente, nuestros resultados están bien ya que la complejidad es igual. Esto lo podemos comprobar con las gráficas que hemos generado y están puestas anteriormente.

QuickSort: Si comparamos nuestros resultados con los que deberían de dar teóricamente podemos concluir que nuestro algoritmo está bien, ya que en las gráficas generadas las dos funciones son iguales.

5.2 Pregunta 2

Debido a la aleatoriedad de las permutaciones median y median_avg tienen la misma gráfica porque es igual de probable que salgan los mismo números en el primero que en el del medio. Sin embargo, median_stat está un poco por debajo debido a que eliges la mediana de tres números, por lo tanto lo normal es que el número elegido sea más cercano a la mediana total y sea más óptimo el algoritmo.

5.3 Pregunta 3

MergeSort: El caso peor es $n \cdot \log(n) + O(n)$, el caso mejor $n \cdot \log(n)$ y el caso medio $n \cdot \log(n)$.

QuickSort: El caso peor es $n^2/2 - n/2$ cuando el array recibido está ordenado y el pivote es el primer elemento, como se puede ver en las gráficas. El caso mejor es $n \cdot \log(n)$.

Para calcular estrictamente cada uno de los casos en el quicksort podemos pasarle como argumento una tabla ordenada (en el caso peor) como hemos hecho. Para el caso mejor habría que elegir permutaciones en las cuales siempre se escoja como pivote la mediana de las tablas. En el caso medio (en los dos algoritmos) cuantas más permutaciones distintas más exacto.

5.4 Pregunta 4

El MergeSort es más efectivo empíricamente del QuickSort, esto se debe a que el caso peor del QuickSort es extremadamente ineficiente. A pesar de que para el caso mejor el QuickSort sea mejor al haber un caso tan malo no merece la pena este algoritmo. En la teoría nos sale lo mismo.

Desde el punto de vista de la gestión de memoria es mejor el QuickSort ya que no utiliza memoria dinámica y el MergeSort si.

6. Conclusiones finales.

Como conclusión podemos decir que ha sido una práctica bastante útil para conocer a fondo los algoritmos que hemos tenido que emplear y para aprender a generar gráficas y medir tiempos de quicksort y mergesort. Las funciones se han implementado con sus debidos controles de errores y hemos intentado que sean lo más eficientes posibles. Los resultados en principio son correctos y las gráficas generadas nos dan bien, ya que las hemos comparado con la gráfica de la función que deberían dar y son iguales. Además hemos aprendido que el algoritmo mergesort puede ser más eficiente cambiando la manera de escoger el pivote.